

Projet d'application Android de réservation pour R3st0.fr

Table des matières

Daily scrum.....	2
Planification.....	3
Lien vers les Git :	3
Ticket N°5 - Itération 2 - Suite maquette app.....	4
Ticket N°6 – Itération 2 – Modification de l’api	4
Ticket N°7 - Itération 2 - Création de la vue "détail"	8
Ticket N°8 - Itération 2 - Connexion avec l'API	10

Daily scrum

17/11 :

Chacun finit ses tâches afin de finir l'itération 1.

24/11 :

Heymad s'est occupé de créer la vue de la fiche détaillée d'un restaurant.

Garence est chargée de créer la partie l'API qui va aller chercher les détails du restaurant comme le nom, l'adresse, la description, la photo, le type de cuisine dans la base de données dans le fichier "getOneById.php".

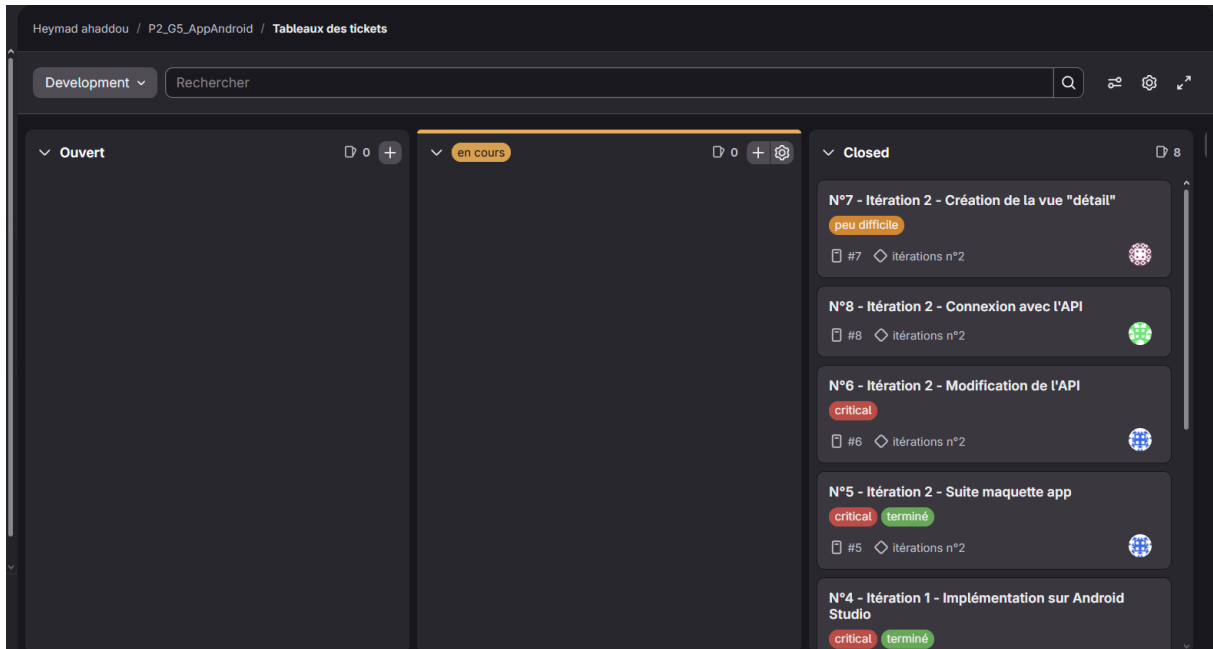
Et la partie qui va aller chercher les notes du restaurant dans la base de données et en faire la moyenne dans le fichier "getNoteMoyenne.php".

Nikauly va s'occuper de modifier la partie de l'API qui permet d'aller chercher les critiques du restaurant dans la base de données dans le fichier "getCritiqueById.php".

Et de faire la connexion de l'API avec l'application.

Planification

Page des tickets :



Lien vers les Git :

Git principal: https://gitlab.com/Hahaddou/p2_g5_appandroid

Git API : <https://gitlab.com/Hahaddou/lienapi>

Ticket N°5 - Itération 2 - Suite maquette app

Cette maquette a été conçue **sur Canva** pour plusieurs raisons stratégiques :



Cette maquette a été faite toujours avec le code couleur du site web.

On peut y voir la fiche détail avec une photo du restaurant, le nom, la ville, ect, et un bouton Réservation qui permettra plus tard la réservation.

Ticket N°6 – Itération 2 – Modification de l'api

Groupe 5 – Nikaully, Garence, Heymad

Itération n°2 – Affichage du détail d'un restaurant

ce ticket concerne la création des endpoints API nécessaires pour récupérer les données détaillées d'un rant et sa note moyenne, qui seront utilisées dans l'application Android.

```
11 $id = intval($_GET['id']);
12
13 try {
14     // Récupérer les infos du restaurant avec la première photo
15     $sql = "SELECT r.*, p.cheminP AS photoR
16           FROM resto r
17           LEFT JOIN photo p ON r.idR = p.idR
18           WHERE r.idR = :id
19           LIMIT 1";
20
21     $stmt = $pdo->prepare($sql);
22     $stmt->execute(['id' => $id]);
23     $resto = $stmt->fetch(PDO::FETCH_ASSOC);
24
25     // Si aucun resto trouvé
26     if (!$resto) {
27         http_response_code(404);
28         echo json_encode(["error" => "Aucun restaurant trouvé pour cet ID"]);
29         exit;
30     }
31
32     // Récupérer toutes les photos du restaurant
33     $sqlPhotos = "SELECT cheminP FROM photo WHERE idR = :id";
34     $stmtPhotos = $pdo->prepare($sqlPhotos);
35     $stmtPhotos->execute(['id' => $id]);
36     $photos = $stmtPhotos->fetchAll(PDO::FETCH_COLUMN);
37     $resto['photos'] = $photos;
38
39     // Récupérer les types de cuisine
40     $sqlTypes = "SELECT tc.id_tc, tc.libelle_tc
41                FROM type_cuisine tc
42                INNER JOIN resto_type_cuisine rtc ON tc.id_tc = rtc.id_tc
43                WHERE rtc.idR = :id";
44     $stmtTypes = $pdo->prepare($sqlTypes);
45     $stmtTypes->execute(['id' => $id]);
46     $types = $stmtTypes->fetchAll(PDO::FETCH_ASSOC);
47     $resto['types'] = $types;
48 }
```

Itération n°2 – Affichage du détail d'un restaurant

```
44
45 $stmtTypes = $pdo->prepare($sqlTypes);
46 $stmtTypes->execute(['id' => $id]);
47 $typesCuisine = $stmtTypes->fetchAll(PDO::FETCH_ASSOC);
48 $resto['types_cuisine'] = $typesCuisine;
49
50 // Récupérer les critiques du restaurant
51 $sqlCritiques = "SELECT
52                 c.idR,
53                 c.idU,
54                 c.note,
55                 c.commentaire,
56                 u.pseudoU AS pseudo_utilisateur,
57                 u.mailU AS email_utilisateur
58                 FROM critiqueur c
59                 INNER JOIN utilisateur u ON c.idU = u.idU
60                 WHERE c.idR = :id AND c.note IS NOT NULL
61                 ORDER BY c.note DESC";
62
63 $stmtCritiques = $pdo->prepare($sqlCritiques);
64 $stmtCritiques->execute(['id' => $id]);
65 $critiques = $stmtCritiques->fetchAll(PDO::FETCH_ASSOC);
66
67 // Calculer les statistiques des critiques
68 $resto['critiques'] = $critiques;
69 $resto['nombre_avis'] = count($critiques);
70
71 if (count($critiques) > 0) {
72     $total = array_sum(array_column($critiques, 'note'));
73     $resto['note_moyenne'] = round($total / count($critiques), 1);
74     $resto['note_min'] = min(array_column($critiques, 'note'));
75     $resto['note_max'] = max(array_column($critiques, 'note'));
76 } else {
77     $resto['note_moyenne'] = 0;
78     $resto['note_min'] = 0;
79     $resto['note_max'] = 0;
80 }
81
82 header('Content-Type: application/json');
83 echo json_encode($resto, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
84
85 } catch (PDOException $e) {
86     http_response_code(500);
87     echo json_encode([
88         "error" => "Erreur de base de données",
89         "message" => $e->getMessage()
90     ]);
91 }
92 ?>
93
```

Ce fichier PHP définit un endpoint d'API qui renvoie en JSON toutes les informations détaillées d'un restaurant (infos de base, photos, types de cuisine, critiques et statistiques sur les notes) à partir de son id passé en paramètre GET.

Voici ce qu'il fait, étape par étape :

1. Vérification du paramètre id

- Il vérifie que `$_GET['id']` existe, sinon il renvoie un code HTTP 400 avec un JSON contenant une erreur, puis arrête le script.
- Il convertit ensuite l'id en entier avec `intval` pour éviter les injections et garantir un entier.

Itération n°2 – Affichage du détail d'un restaurant

2. Récupération des infos du restaurant

- Il exécute une requête SQL avec préparation pour récupérer les colonnes de la table resto (r.*) et la première photo liée (p.cheminP AS photoR) via un LEFT JOIN sur la table photo, filtrée par r.idR = :id et LIMIT 1.
- Si aucun restaurant n'est trouvé, il renvoie un code HTTP 404 avec un message d'erreur JSON.

3. Récupération de toutes les photos

- Il exécute une deuxième requête SELECT cheminP FROM photo WHERE idR = :id.
- fetchAll(PDO::FETCH_COLUMN) retourne un tableau simple des chemins d'images, qui est ajouté au tableau \$resto sous la clé 'photos'.

4. Récupération des types de cuisine

- Il fait un INNER JOIN entre type_cuisine et resto_type_cuisine pour obtenir tous les types de cuisine associés à ce restaurant.
- Le résultat (id_tc et libelle_tc) est ajouté à \$resto sous la clé 'types_cuisine'.

5. Récupération des critiques

- Il sélectionne les critiques dans la table critiquer, joint la table utilisateur pour récupérer le pseudo et l'email de l'utilisateur, filtre sur c.idR = :id et c.note IS NOT NULL, et ordonne par la note décroissante.
- Le tableau de critiques (note, commentaire, pseudo_utilisateur, email_utilisateur, etc.) est ajouté à \$resto sous la clé 'critiques'.

6. Calcul des statistiques sur les avis

- Il compte le nombre de critiques avec count(\$critiques) et stocke le résultat dans \$resto['nombre_avis'].
- S'il y a au moins un avis :
 - Il somme toutes les notes (array_sum(array_column(\$critiques, 'note')))
 - Calcule la moyenne, arrondie à une décimale, dans \$resto['note_moyenne']
 - Trouve la note minimale et maximale avec min(...) et max(...), stockées dans 'note_min' et 'note_max'.
- S'il n'y a aucun avis, il met ces trois valeurs à 0.

7. Réponse JSON ou erreur serveur

Itération n°2 – Affichage du détail d'un restaurant

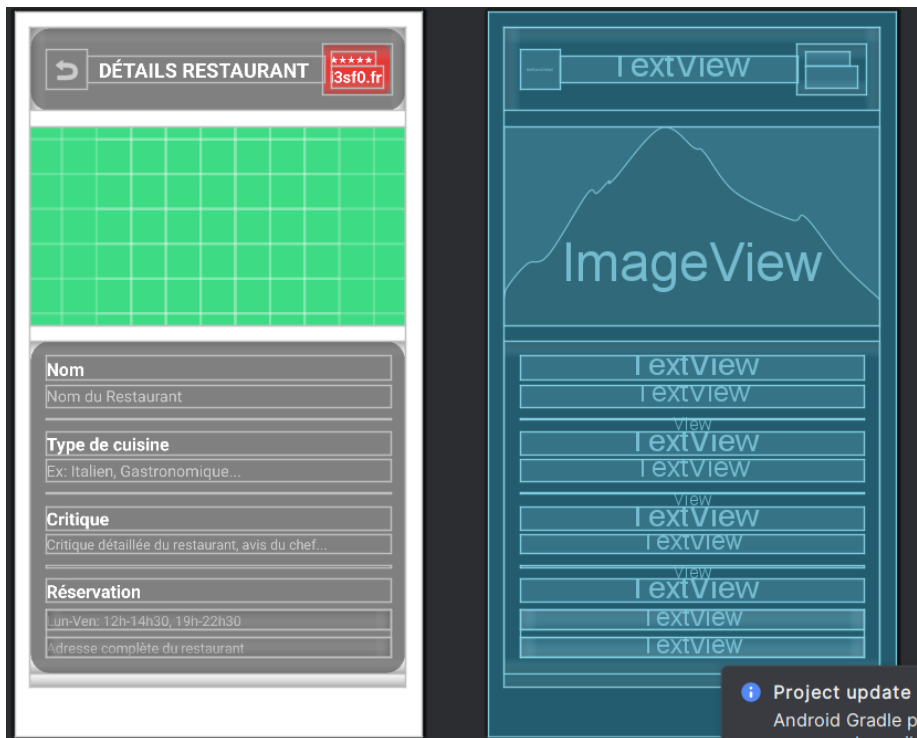
- Si tout se passe bien, il définit l'en-tête Content-Type: application/json et renvoie `json_encode($resto, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT)`.
- En cas d'exception PDO, il renvoie un code HTTP 500 avec un JSON contenant "Erreur de base de données" et le message de l'exception.

Ticket N°7 - Itération 2 - Création de la vue "détail"

Cette vue affiche les informations complètes d'un restaurant sélectionné depuis la liste principale.

L'interface suit une structure linéaire et scrollable, avec une hiérarchie visuelle claire et une séparation nette des sections.

Le design privilégie la lisibilité, la cohérence visuelle et l'expérience utilisateur sur Android.



Structure du layout (activity_detail.xml)

1. Conteneur principal

- ScrollView : permet de défiler le contenu si l'écran est trop petit.
- LinearLayout vertical à l'intérieur : organise tous les éléments de haut en bas.

2. En-tête

- Contient un bouton de retour (←), le titre “**DÉTAILS RESTAURANT**”, et un badge avec les étoiles (★★★★★) + le site “[i3sf0.fr](#)”.
- Le bouton btnBackDetail permet de revenir à l'écran précédent.
- Le badge utilise un fond rouge arrondi pour attirer l'attention sur la note.

3. Image principale

- ImageView de hauteur fixe (200dp) avec un scaleType="centerCrop" pour couvrir l'espace sans déformation.
- L'image est pour l'instant une placeholder (ic_launcher_background), mais sera remplacée dynamiquement.

4. Section regroupée des infos

- Un LinearLayout vertical avec fond gris arrondi, qui contient 4 blocs :
 - **Nom** : titre + valeur (ex: “Nom du Restaurant”).
 - **Type de cuisine** : titre + valeur (ex: “Italien, Gastronomique...”).
 - **Critique** : titre + texte plus long, avec un interligne augmenté pour la lisibilité.
 - **Réservation** : divisé en **horaires** et **adresse**, affichés sur deux lignes distinctes.
- Chaque bloc est séparé par un View de 1dp de couleur #555555 (séparateur discret).

Ticket N°8 - Itération 2 - Connexion avec l'API



Itération n°2 – Affichage du détail d'un restaurant

Code :

```
1 usage
private void initView() {
    textRestaurantName = findViewById(R.id.textRestaurantName);
    textRestaurantCuisine = findViewById(R.id.textRestaurantCuisine);
    textRestaurantCritique = findViewById(R.id.textRestaurantCritique);
    textRestaurantHours = findViewById(R.id.textRestaurantHours);
    textRestaurantAddress = findViewById(R.id.textRestaurantAddress);
    imageRestaurantDetail = findViewById(R.id.imageRestaurantDetail);
    btnBackDetail = findViewById(R.id.btnBackDetail);
}

// Charger une image depuis assets
1 usage
private void loadImageFromAssets(String imageName) {
    try {
        InputStream inputStream = getAssets().open(imageName);
        Bitmap bitmap = BitmapFactory.decodeStream(inputStream);
        imageRestaurantDetail.setImageBitmap(bitmap);
        inputStream.close();
    } catch (IOException e) {
        e.printStackTrace();
        imageRestaurantDetail.setImageResource(R.drawable.ic_launcher_background);
    }
}
```

Recuperation de les id pour pouvoir afficher les informations

Ainsi que le changement des images depuis le dossier asset

Formatage des horaires pour que ce soit plus beau a voir

```
// Methode pour formater les horaires de maniere lisible
1 usage
private String simplifierHoraires(String html) {
    if (html == null || html.isEmpty() || html.equals("null")) {
        return "🕒 Horaires non disponibles";
    }

    // Créer un format lisible à partir du HTML
    StringBuilder formatted = new StringBuilder("🕒 HORAIRES\n\n");

    // Extraire les informations principales
    if (html.contains("Midi")) {
        formatted.append("🕒 MIDI\n");
        formatted.append("Semaine : 11h45 - 14h30\n");
        formatted.append("Week-end : 11h45 - 15h00\n");
    }

    if (html.contains("Soir")) {
        formatted.append("🌙 SOIR\n");
        formatted.append("Semaine : 18h45 - 22h30\n");
        formatted.append("Week-end : 18h45 - 01h00\n");
    }

    if (html.contains("emporter") || html.contains("À emporter")) {
        formatted.append("🍷 À EMPORTER\n");
        formatted.append("Semaine : 11h30 - 23h00\n");
        formatted.append("Week-end : 11h30 - 02h00\n");
    }

    return formatted.toString();
}
```

Groupe 5 – Nikauy, Garence, Heymad

Itération n°2 – Affichage du détail d'un restaurant

Récupération des données depuis l'api rest

```
1 usage
private class FetchRestaurantDetailTask extends AsyncTask<String, Void, JSONObject>

    @Override
    protected JSONObject doInBackground(String... urls) {
        try {
            URL url = new URL(urls[0]);
            HttpURLConnection connection = (HttpURLConnection) url.openConnection();
            connection.setRequestMethod("GET");
            connection.setConnectTimeout(10000);
            connection.setReadTimeout(10000);

            int responseCode = connection.getResponseCode();

            if (responseCode == HttpURLConnection.HTTP_OK) {
                BufferedReader reader = new BufferedReader(
                    new InputStreamReader(connection.getInputStream())
                );
                StringBuilder response = new StringBuilder();
                String line;

                while ((line = reader.readLine()) != null) {
                    response.append(line);
                }
                reader.close();
                connection.disconnect();

                return new JSONObject(response.toString());
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

Itération n°2 – Affichage du détail d'un restaurant

Affichage des informations dans notre fichiers xml

```
protected void onPostExecute(JSONObject result) {
    if (result != null) {
        try {
            // Vérifier s'il y a une erreur
            if (result.has(name: "error")) {
                Toast.makeText(context: RestoDetailActivity.this,
                    result.getString(name: "error"), Toast.LENGTH_SHORT).show();
                finish();
                return;
            }

            // 1. Nom du restaurant
            String nom = result.optString(name: "nomR", fallback: "Restaurant");
            textRestaurantName.setText(nom);

            // 2. Adresse complète
            String numAdr = result.optString(name: "numAdrR", fallback: "");
            String voieAdr = result.optString(name: "voieAdrR", fallback: "");
            String cp = result.optString(name: "cpR", fallback: "");
            String ville = result.optString(name: "villeR", fallback: "");
            String adresse = numAdr + " " + voieAdr + ", " + cp + " " + ville;
            textRestaurantAddress.setText(adresse.trim());

            // 3. Types de cuisine
            if (result.has(name: "types_cuisine")) {
                JSONArray typesCuisine = result.getJSONArray(name: "types_cuisine");
                StringBuilder cuisines = new StringBuilder();

                for (int i = 0; i < typesCuisine.length(); i++) {
                    JSONObject type = typesCuisine.getJSONObject(i);
                    cuisines.append(type.getString(name: "libelle_tc"));
                    if (i < typesCuisine.length() - 1) {
                        cuisines.append(", ");
                    }
                }
            }
        }
    }
}
```

Itération n°2 – Affichage du détail d'un restaurant

Amélioration de l'affichage de la note moyenne et des critiques pour que ce soit plus beau à voir.

```
// Afficher la note moyenne
if (nombreAvis > 0) {
    texteCritiques.append("★ Note moyenne: ")
        .append(noteMoyenne)
        .append("/5 (")
        .append(nombreAvis)
        .append(" avis)\n\n");

    // Afficher chaque critique
    for (int i = 0; i < critiques.length(); i++) {
        JSONObject critique = critiques.getJSONObject(i);
        int note = critique.optInt( name: "note", fallback: 0);
        String pseudo = critique.optString( name: "pseudo_utilisateur", fallback: "Anonyme");
        String commentaire = critique.optString( name: "commentaire", fallback: "");

        // Afficher les étoiles
        for (int j = 0; j < note; j++) {
            texteCritiques.append("★");
        }
        for (int j = note; j < 5; j++) {
            texteCritiques.append("☆");
        }

        texteCritiques.append(" - ").append(pseudo);

        // Afficher le commentaire si présent
        if (!commentaire.isEmpty() && !commentaire.equals("null")) {
            texteCritiques.append("\n").append(commentaire);
        }

        if (i < critiques.length() - 1) {
            texteCritiques.append("\n\n");
        }
    }
}
```

Itération n°2 – Affichage du détail d'un restaurant

Resultat final : https://gitlab.com/Hahaddou/p2_g5_appandroid/-/tree/iteration-n%C2%B02?ref_type=heads

Lien gitlab iteration n°2 :



Groupe 5 – Nikaully, Garence, Heymad